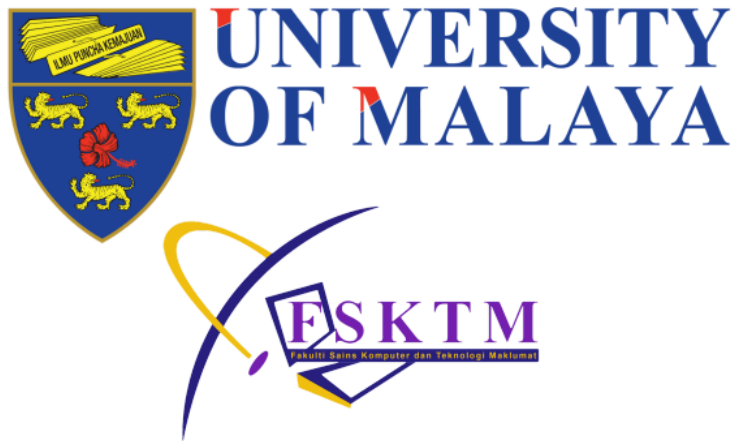




WIA1002 DATA STRUCTURE SEMESTER 2, 2025/2026

**Topic: Smart Library Project
(Console-Based Library Management System)**

Lecturer's Name: Dr. Zainab Malik

Occurence: 7

Group: 8

Group Member:

No.	Name	Matric Number
1	TAN CHEE KEAT	25006123
2	LIM HONG ZHANG	25006100
3	LEE MING DAO	25006825
4	JITESH A/L MOGANA RAJA	25006745
5	TEH XU ZHE	25006355

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. System Design & Approach
5. Data Structures Used
6. Implementation Details
7. User Interface & Workflow
8. Data Storage
9. Testing
10. Conclusion
11. Appendices

1. Introduction

SmartLibrary is a Java-based console application that simulates a small-scale library management system. It supports two user roles — **Admin** and **Student** — each with a dedicated set of operations. The application was designed to efficiently managing a book catalogue and tracking student borrowing activity.

The system stores its book catalogue in a **Binary Search Tree (BST)** keyed by ISBN, enabling efficient ordered retrieval, search, and modification. Each student's borrowing history is maintained in a **linked-list stack**, which naturally presents the most recent activity first. All data is persisted to local text files, allowing the application to restore its state across sessions without requiring a database.

2. Problem Statement

Small libraries and academic institutions often lack access to commercial library management software, leading to several challenges:

1. **Inefficient Book Lookup:** Searching for a specific book in an unorganized collection requires scanning every record, resulting in $O(n)$ lookup time.
2. **No Borrowing History:** Without systematic tracking, it is difficult to determine which books a student has borrowed, when they were borrowed, and whether they have been returned.
3. **Lack of Role Separation:** Library staff and students often need different levels of access. Staff should manage the catalogue, while students should only borrow and return books.
4. **Data Loss Between Sessions:** Without persistence, all catalogue and borrowing data is lost when the application terminates.
5. **No Sorted Display:** Users benefit from seeing books in a consistent, sorted order, but maintaining sort order manually is error-prone.

The SmartLibrary project addresses these problems by implementing a role-based library management system that uses purpose-built data structures to deliver efficient search, insertion, and deletion of book records, along with chronological tracking of every borrowing transaction.

3. Objectives

1. Design and implement a **Binary Search Tree** to store and manage book records, enabling $O(\log n)$ average-case search, insertion, and deletion by ISBN.
2. Design and implement a **linked-list stack** for each student to record borrowing history in last-in-first-out (LIFO) order.
3. Implement a **role-based access control** system separating Admin and Student operations.
4. Implement **file-based data persistence** so that all catalogue, loan, and user data survives application restarts.
5. Provide a **clean console-based user interface** with input validation, formatted table output, and clear error messages.
6. Apply **abstraction** through a Java interface (`LibraryADT`) to separate the service layer from the UI.

4. System Design & Approach

4.1 Architectural Overview

The application follows a **layered architecture** with clear separation of concerns:

LAYER	COMPONENT(S)	RESPONSIBILITY
Presentation	Main.java	Console menus, user input/output, table formatting
Service	SmartLibrary.java UserStore.java	Business logic, data validation, persistence orchestration
ADT/Interface	LibraryADT.java	Contract for library operations (abstraction layer)
Data Structures	BookBST.java BorrowHistoryStack.java	Custom BST and Stack implementations
Models	Book.java , LoanRecord.java , User.java	Data carriers (POJOs)
Persistence	smart_library_data.txt user_info.txt	Data storage

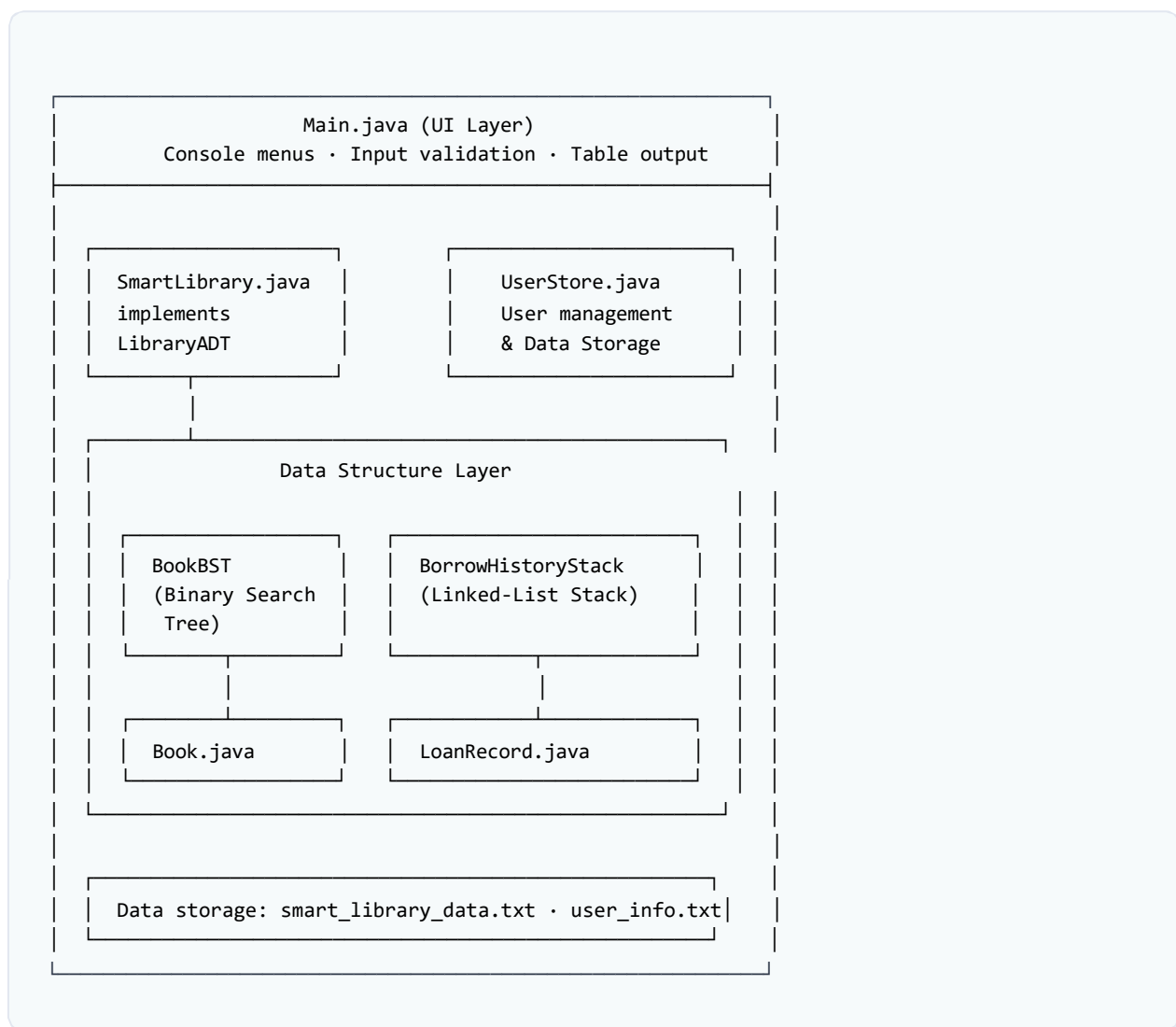


Figure 1: System Architecture Diagram

4.2 Design Decisions

Why a Binary Search Tree for the Catalogue?

A BST was chosen over a simple list or array because the library needs to frequently search for books by ISBN, add new books, and remove books. A BST provides $O(\log n)$ average-case performance for all three operations, compared to $O(n)$ for a linear scan. Additionally, an in-order traversal of the BST naturally produces a sorted list of books, which is required for the "View All Available Books" feature.

Why a Stack for Borrowing History?

A stack (LIFO) was chosen for borrowing history because users most naturally want to see their *most recent* activity first. Each new borrow or return event is pushed onto the stack, and when the history is displayed, it reads from top to bottom which is newest to oldest.

Why Local File Storage?

The project uses local text files rather than a database to keep the application simple and dependency-free. The custom escape-character parser ensures that special characters in book titles and author names do not corrupt the data format.

Why an Interface (LibraryADT)?

The `LibraryADT` interface abstracts the library operations so the UI layer (`Main.java`) depends only on the contract, not on the implementation. This allows the underlying data structures to be changed without modifying the UI code.

5. Data Structures Used

5.1 Binary Search Tree — BookBST

The BookBST class implements a standard binary search tree where each node contains a Book object. The tree is keyed by the book's ISBN number (a long value).

Node Structure

```
package smartlibrary;

import java.util.ArrayList;
import java.util.List;

/**
 * Binary search tree that stores all currently available books.
 * Books are ordered by ISBN so search, insert, and delete can follow the tree
 * instead of scanning every book in the catalogue.
 */
public class BookBST {
    private Node root;

    //One node in the binary search tree.
    private static class Node {
        private Book book;
        private Node left;
        private Node right;

        private Node(Book book) {
            this.book = book;
        }
    }
}
```

Operations & Complexity

OPERATION	METHOD SIGNATURE	AVERAGE CASE	WORST CASE
Insert	insert(Book book)	O(log n)	O(n)
Search	search(long isbn)	O(log n)	O(n)
Delete	deleteByIsbn(long isbn)	O(log n)	O(n)
In-order traversal	toList()	O(n)	O(n)

Deletion Strategy: When a node with two children is deleted, it is replaced by its *in-order successor* — the smallest node in its right subtree. This preserves the BST ordering invariant.

BST Traversal Diagram

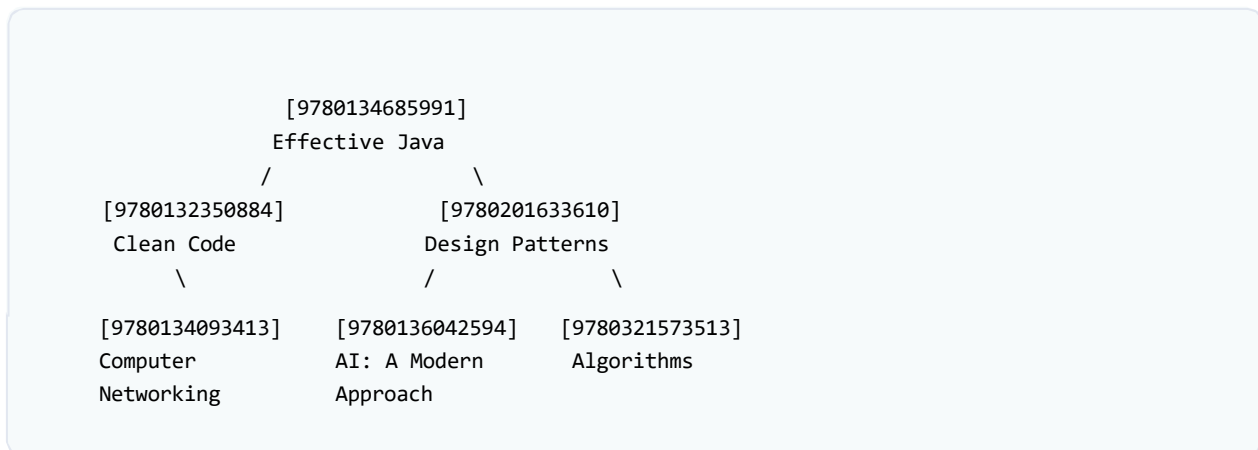


Figure 2: Example BST structure (keyed by ISBN, in-order traversal yields sorted output)

5.2 Linked-List Stack — BorrowHistoryStack

Node Structure

```
package smartlibrary;

import java.util.ArrayList;
import java.util.List;

/**
 * Stack that stores borrowing history for one student.
 * New loan records are pushed to the top, so history is displayed with the most
 * recent activity first.
 */
public class BorrowHistoryStack {
    private Node top;

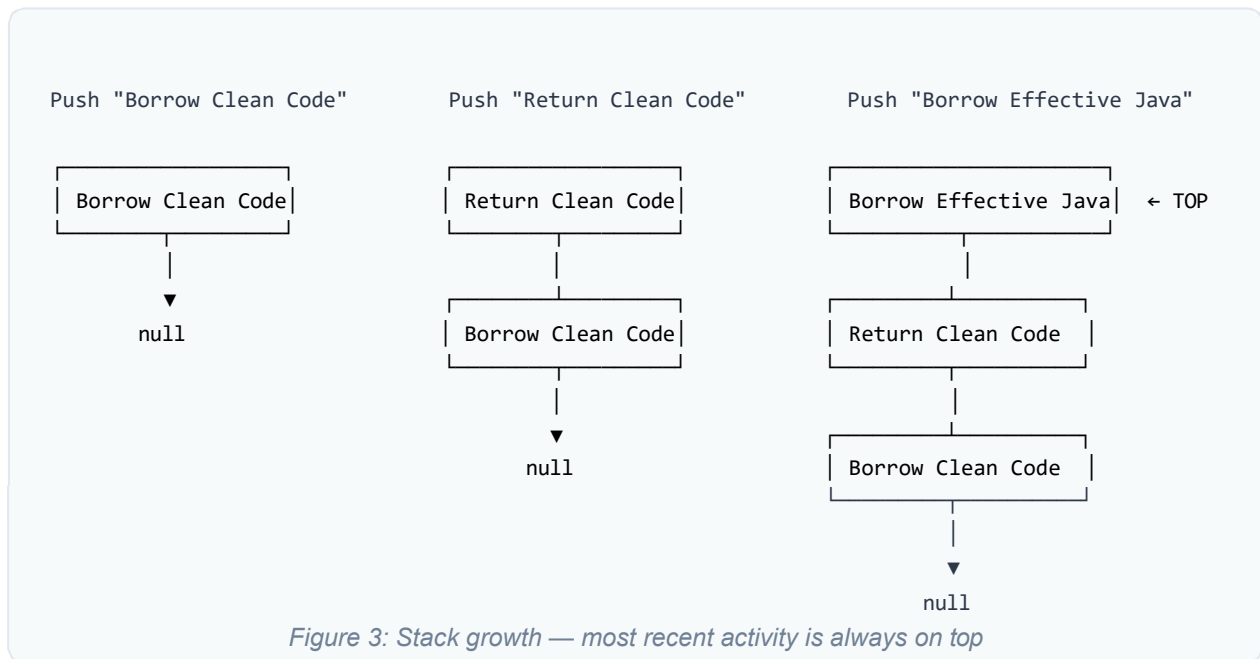
    //One linked-list node used by the stack.
    private static class Node {
        private final LoanRecord loanRecord;
        private final Node next;

        private Node(LoanRecord loanRecord, Node next) {
            this.loanRecord = loanRecord;
            this.next = next;
        }
    }
}
```

Operations & Complexity

OPERATION	METHOD SIGNATURE	TIME COMPLEXITY
Push	<code>push(LoanRecord loanRecord)</code>	O(1)
Is Empty	<code>isEmpty()</code>	O(1)
Find Active Loan	<code>findActiveLoanByIsbn(long isbn)</code>	O(n)
Convert to List	<code>toList()</code>	O(n)

Stack Behaviour Diagram



5.3 LinkedHashMap — Student-to-Stack Mapping

A `LinkedHashMap<String, BorrowHistoryStack>` maps each student ID to their personal history stack. This provides:

- **$O(1)$ lookup** to retrieve a student's history stack by ID
- **Insertion-order iteration** for consistent, reproducible file output
- **Lazy initialization** via `computeIfAbsent()`

5.4 Summary of Data Structures

DATA STRUCTURE	CLASS	PURPOSE	KEY ADVANTAGE
Binary Search Tree	<code>BookBST</code>	Book catalogue storage	$O(\log n)$ search/insert/delete; sorted traversal
Linked-List Stack	<code>BorrowHistoryStack</code>	Per-student loan history	$O(1)$ push; natural LIFO ordering
LinkedHashMap	(JDK built-in)	Student → Stack mapping	$O(1)$ lookup; insertion-order iteration
ArrayList	(JDK built-in)	Temporary lists, file I/O	Dynamic sizing; indexed access

6. Implementation Details

6.1 File Descriptions

FILE	LINES	ROLE
Main.java	525	Console UI — entry point, menus, input reading, formatted table output
SmartLibrary.java	424	Service layer — implements LibraryADT, manages BST and stacks, handles file I/O
LibraryADT.java	53	Interface — defines 9 core library operations
BookBST.java	162	BST — insert, search, delete, in-order traversal
BorrowHistoryStack.java	71	Stack — push, isEmpty, findActiveLoan, toList
Book.java	37	Model — immutable book record (ISBN, title, author)
LoanRecord.java	67	Model — borrowing transaction (student, book, dates, status)
User.java	29	Model — registered user (ID, name, role)
UserStore.java	215	User management — registration, lookup, persistence

Total: ~1,583 lines of Java code across 9 source files.

6.2 Key Algorithms

BST Insertion (Recursive)

New books are inserted by recursively traversing the tree: if the ISBN is less than the current node, go left; if greater, go right; if a `null` child is reached, insert the new node. Duplicate ISBNs are rejected by returning `false`.

BST Deletion (In-order Successor)

When deleting a node with two children, the algorithm finds the in-order successor (the smallest node in the right subtree), copies its book data into the current node, and then recursively deletes the successor. This maintains the BST invariant without restructuring the entire tree.

Borrowing Workflow

1. Search the BST for the requested ISBN.
2. If found, remove the book from the BST (it is no longer available).
3. Create a `LoanRecord` with today's date and a 14-day due date.
4. Push the loan record onto the student's history stack.
5. Save all data to file.

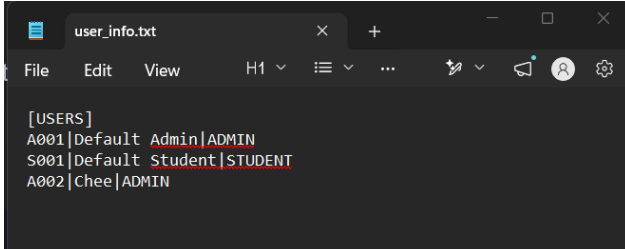
Return Workflow

1. Look up the student's history stack.
2. Search the stack for an active (unreturned) loan matching the ISBN.
3. If found, mark the loan as returned.
4. Re-insert the book into the BST (it is available again).
5. Save all data to file.

6.3 Data Persistence Format

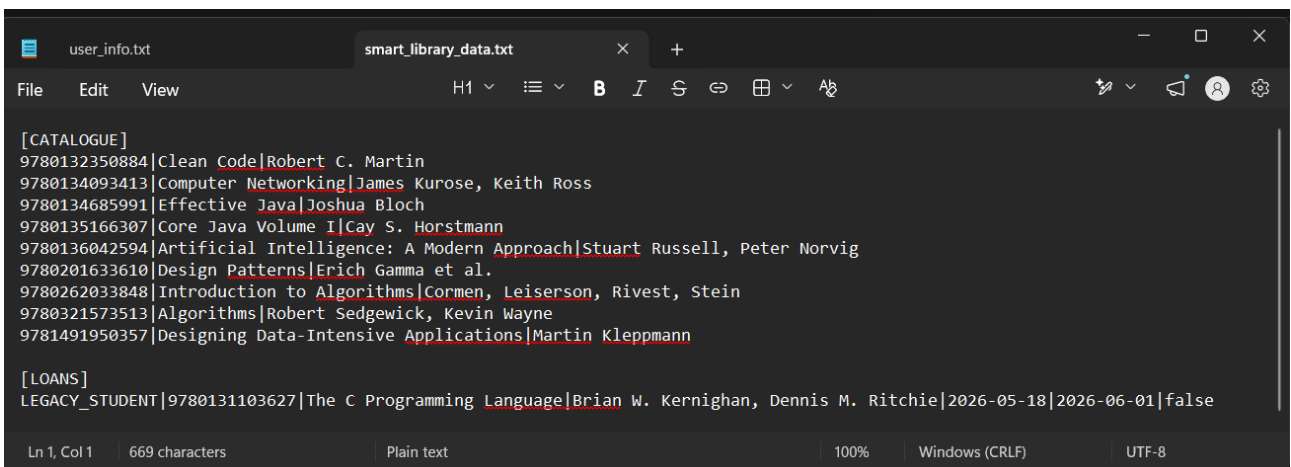
Data is stored in pipe-delimited text files with escape character support:

user_info.txt



```
[USERS]
A001|Default Admin|ADMIN
S001|Default Student|STUDENT
A002|Chee|ADMIN
```

smart_library_data.txt



```
[CATALOGUE]
9780132350884|Clean Code|Robert C. Martin
9780134093413|Computer Networking|James Kurose, Keith Ross
9780134685991|Effective Java|Joshua Bloch
9780135166307|Core Java Volume I|Cay S. Horstmann
9780136042594|Artificial Intelligence: A Modern Approach|Stuart Russell, Peter Norvig
9780201633610|Design Patterns|Erich Gamma et al.
9780262033848|Introduction to Algorithms|Cormen, Leiserson, Rivest, Stein
9780321573513|Algorithms|Robert Sedgewick, Kevin Wayne
9781491950357|Designing Data-Intensive Applications|Martin Kleppmann

[LOANS]
LEGACY_STUDENT|9780131103627|The C Programming Language|Brian W. Kernighan, Dennis M. Ritchie|2026-05-18|2026-06-01|false
```

Escape Handling: The custom `splitLine()` parser handles backslash-escaped pipe characters (`\|`) and escaped backslashes (`\\`), ensuring that special characters in book titles and author names do not break the data format.

```
private List<String> splitLine(String line) {
    List<String> parts = new ArrayList<>();
    StringBuilder current = new StringBuilder();
    boolean escaped = false;

    for (int index = 0; index < line.length(); index++) {
        char character = line.charAt(index);

        if (escaped) {
            current.append(character);
            escaped = false;
        } else if (character == '\\') {
            escaped = true;
        } else if (character == '|') {
            parts.add(current.toString());
            current.setLength(0);
        } else {
            current.append(character);
        }
    }

    if (escaped) {
        current.append('\\');
    }

    parts.add(current.toString());
    return parts;
}

//Escapes characters that would otherwise break the pipe-delimited file.
private String escape(String value) {
    return value.replace("\\", "\\\\").replace("|", "\\|");
}
```

7. User Interface & Workflow

7.1 Application Flow

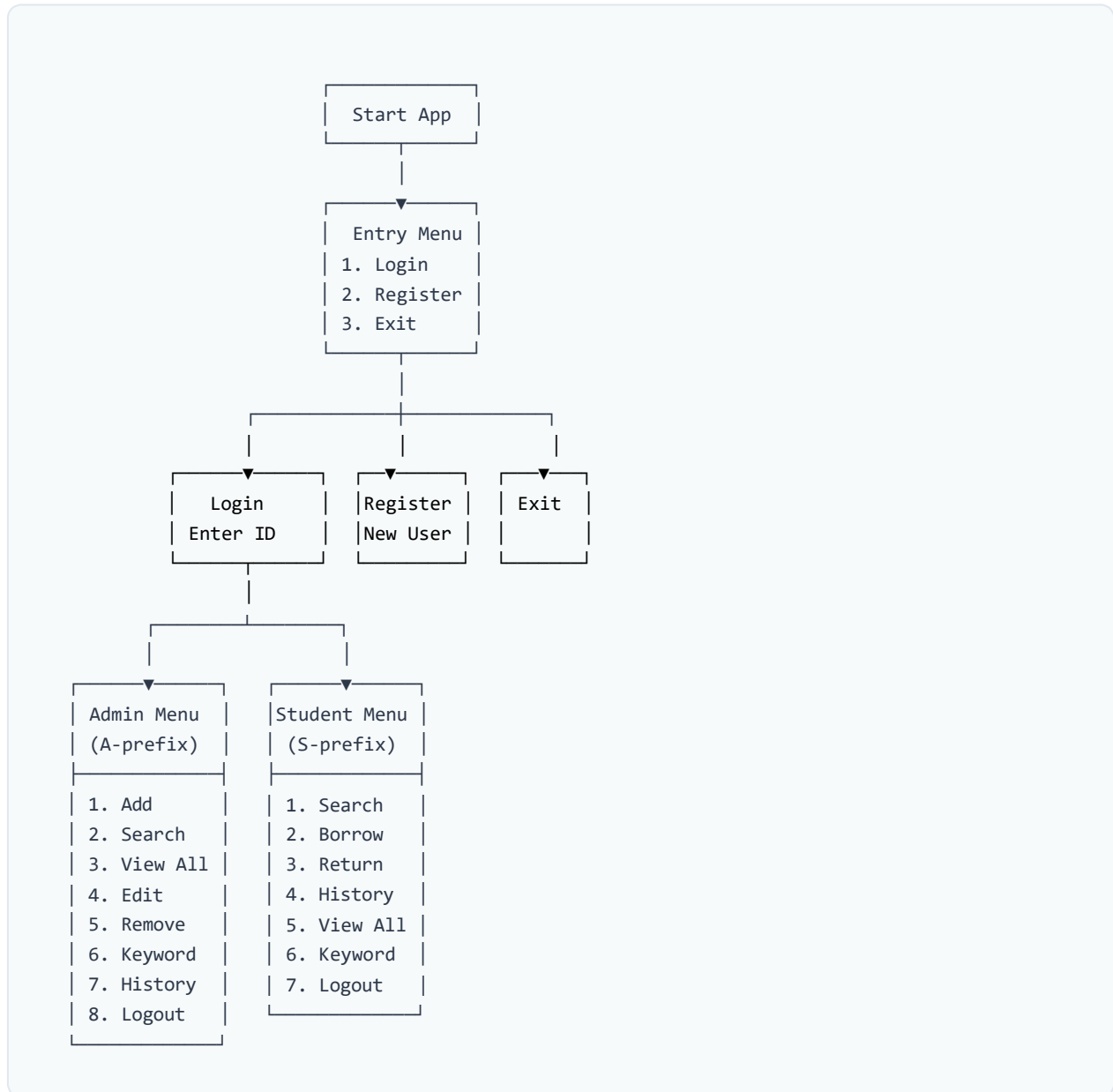


Figure 4: Application Navigation Flow

7.2 Console Output Formatting

The UI layer dynamically calculates column widths based on the actual data content, ensuring that tables are properly aligned regardless of title or author length. This is implemented in the `printBooks()` and `printLoanRecords()` methods using `String.format()` with computed width specifiers.

7.3 Input Validation

VALIDATION	METHOD	BEHAVIOUR
Empty text fields	<code>readRequiredText()</code>	Returns <code>null</code> and prints error
Invalid ISBN format	<code>readPositiveIsbn()</code>	Rejects non-numeric or negative values
Invalid menu choice	<code>readMenuChoice()</code>	Returns -1, triggering the default branch
Invalid user ID format	<code>isValidUserId()</code>	Must start with 'A' or 'S'
Duplicate user ID	<code>registerUser()</code>	Checks map before insertion
Duplicate ISBN	<code>BookBST.insert()</code>	Returns <code>false</code> for existing ISBNs

8. Data Storage

8.1 Persistence Strategy

The application uses a **write-on-change** strategy. The entire data file is rewritten after every state-changing operation (add, edit, remove, borrow, return, register). While this is less efficient than incremental updates, it ensures data consistency and simplifies the implementation for a small-scale application.

8.2 Load Process

1. Check if the data file exists. If not, start with an empty catalogue.
2. Read all lines and parse section headers (`[CATALOGUE]` , `[LOANS]`).
3. Parse book records into `Book`
4. Parse loan records into `LoanRecord` objects and insert into the BST. objects, grouped by student ID.
5. Push loan records into per-student stacks in reverse order to preserve the original stack ordering.

8.3 Legacy Data Migration

The system supports an older `[BORROW_HISTORY]` section format. Records found under this header are automatically migrated to the modern `[LOANS]` format under a special `LEGACY_STUDENT` ID. This ensures backward compatibility when upgrading from older versions.

9. Testing

9.1 Testing Approach

Testing was conducted through **manual functional testing** of all user-facing operations. Each test case was designed to verify both the expected (happy path) and error (edge case) behaviour of the system.

9.2 Functional Test Cases

#	TEST CASE	STEPS	EXPECTED RESULT	STATUS
1	Add a new book	Login as admin → Add Book → Enter valid ISBN, title, author	"Book added successfully." Book appears in catalogue.	✓ Pass
2	Add duplicate ISBN	Add a book with an ISBN already in the catalogue	"Book could not be added." Catalogue unchanged.	✓ Pass
3	Search book by ISBN	Enter a valid ISBN present in the catalogue	Book details displayed in a formatted table.	✓ Pass
4	Search non-existent ISBN	Enter an ISBN not in the catalogue	"No available book found." message displayed.	✓ Pass
5	View all available books	Select "View All Available Books" from menu	All books displayed sorted by ISBN in formatted table.	✓ Pass
6	Edit book details	Login as admin → Edit → Enter ISBN → Enter new title/author	"Book details updated successfully." Updated values appear.	✓ Pass
7	Remove a book	Login as admin → Remove → Enter ISBN	"Book removed permanently." Book no longer in catalogue.	✓ Pass
8	Borrow a book	Login as student → Borrow → Enter available ISBN	"Borrowed successfully." Book removed from available list.	✓ Pass

#	TEST CASE	STEPS	EXPECTED RESULT	STATUS
9	Borrow unavailable book	Try to borrow a book already checked out	"Borrow failed." message displayed.	✓ Pass
10	Return a book	Login as student → Return → Enter borrowed ISBN	"Returned successfully." Book restored to catalogue.	✓ Pass
11	Return a non-borrowed book	Try to return a book not in the student's active loans	"Return failed." message displayed.	✓ Pass
12	View borrowing history	Login as student → View My History	Loan records displayed most-recent-first.	✓ Pass
13	Search by title keyword	Enter partial title (e.g., "Java")	All matching available books displayed.	✓ Pass
14	Search by author keyword	Enter partial author name	All matching available books displayed.	✓ Pass
15	Data persistence	Add books → Exit → Restart application	Previously added books and loans are restored.	✓ Pass

9.3 Edge Case & Validation Tests

#	SCENARIO	INPUT	EXPECTED BEHAVIOUR	STATUS
1	Empty ISBN field	(blank)	"Invalid ISBN. Please enter numbers only."	✓ Pass
2	Negative ISBN	-12345	"ISBN must be a positive number."	✓ Pass
3	Non-numeric ISBN	abc	"Invalid ISBN. Please enter numbers only."	✓ Pass
4	Empty title/author	(blank)	"This field cannot be empty."	✓ Pass
5	Invalid menu choice	99 or "abc"	"Invalid choice. Please select a number from 1 to N."	✓ Pass
6	Register duplicate user ID	A001 (already exists)	"Registration failed. The user ID may already exist."	✓ Pass

#	SCENARIO	INPUT	EXPECTED BEHAVIOUR	STATUS
7	Login with unregistered ID	S999	"User ID not found. Please register first."	✓ Pass
8	Invalid user ID prefix	X001	"Registration failed. Admin IDs must start with A..."	✓ Pass
9	Edit borrowed book	ISBN of a checked-out book	"Edit failed. Only available catalogue books can be edited."	✓ Pass
10	Corrupt data file	Malformed lines in data file	Invalid lines silently skipped; valid data loaded.	✓ Pass
11	Missing data file	Delete data files → start app	Default users created; empty catalogue initialized.	✓ Pass
12	Pipe character in book title	Title containing " "	Character properly escaped; data saved/loaded correctly.	✓ Pass

10. Conclusion

The SmartLibrary project successfully demonstrates how fundamental data structures can be applied to build a practical, working application. The **Binary Search Tree** provides efficient ordered storage and retrieval of book records, while the **linked-list stack** naturally organizes borrowing history in chronological order. The use of a Java interface (`LibraryADT`) to abstract the library operations promotes clean separation between the UI and business logic layers.

Through this project, the following data structure concepts were applied in practice:

- **Binary Search Tree** — insertion, search, deletion (with in-order successor), and in-order traversal
- **Stack (LIFO)** — push, linked-list traversal, and conversion to list
- **Hash Map** — constant-time key-based lookup for student history mapping
- **Abstract Data Type** — interface-based design for decoupling implementation from contract

11. Appendices

Appendix A: LibraryADT Interface

```
1 package smartlibrary;
2
3 import java.util.List;
4
5 /**
6  * Defines the main operations supported by the Smart Library system.
7  * The implementation hides the data structures used to manage books and loans.
8  */
9 public interface LibraryADT {
10     // Adds a new available book to the catalogue.
11     boolean addBook(long isbn, String title, String author);
12
13     // Finds an available book by ISBN.
14     Book searchBook(long isbn);
15
16     //Borrows a book for a student and removes it from the available catalogue.
17     LoanRecord borrowBook(String studentId, long isbn);
18
19     //Returns a student's active loan and makes the book available again.
20     LoanRecord returnBook(String studentId, long isbn);
21
22     //Lists one student's borrowing history with the newest record first.
23     List<LoanRecord> viewBorrowHistory(String studentId);
24
25     //Lists every book currently available in the catalogue.
26     List<Book> viewAvailableBooks();
27
28     //Updates the title and author of an available catalogue book.
29     boolean editBook(long isbn, String title, String author);
30
31     //Removes an available book from the catalogue.
32     Book removeBook(long isbn);
33
34     //Searches available books by a title or author keyword.
35     List<Book> searchByTitleOrAuthor(String keyword);
36 }
```

Appendix B: BookBST Core Methods

Insertion

```
public boolean insert(Book book) {
    if (root == null) {
        root = new Node(book);
        return true;
    }

    return insertRecursive(root, book);
}

/**
 * Recursively follows the left or right branch based on the ISBN value.
 */
private boolean insertRecursive(Node current, Book book) {
    if (book.getIsbn() == current.book.getIsbn()) {
        return false;
    }

    if (book.getIsbn() < current.book.getIsbn()) {
        if (current.left == null) {
            current.left = new Node(book);
            return true;
        }
        return insertRecursive(current.left, book);
    }

    if (current.right == null) {
        current.right = new Node(book);
        return true;
    }
    return insertRecursive(current.right, book);
}
```

Search

```
public Book search(long isbn) {
    return searchRecursive(root, isbn);
}

private Book searchRecursive(Node current, long isbn) {
    if (current == null) {
        return null;
    }

    if (isbn == current.book.getIsbn()) {
        return current.book;
    }

    if (isbn < current.book.getIsbn()) {
        return searchRecursive(current.left, isbn);
    }

    return searchRecursive(current.right, isbn);
}
```

Delete

```
public boolean deleteByIsbn(long isbn) {
    if (search(isbn) == null) {
        return false;
    }

    root = deleteRecursive(root, isbn);
    return true;
}

private Node deleteRecursive(Node current, long isbn) {
    if (current == null) {
        return null;
    }

    if (isbn < current.book.getIsbn()) {
        current.left = deleteRecursive(current.left, isbn);
        return current;
    }

    if (isbn > current.book.getIsbn()) {
        current.right = deleteRecursive(current.right, isbn);
        return current;
    }

    if (current.left == null) {
        return current.right;
    }

    if (current.right == null) {
        return current.left;
    }

    // For a node with two children, replace it with the smallest book from
    // the right subtree so the BST ordering remains valid.
    Node successor = findSmallest(current.right);
    current.book = successor.book;
    current.right = deleteRecursive(current.right, successor.book.getIsbn());
    return current;
}
```

Travesal

```
public List<Book> toList() {
    List<Book> books = new ArrayList<>();
    addBooksInOrder(root, books);
    return books;
}

/**
 * In-order traversal visits left branch, current book, then right branch.
 */
private void addBooksInOrder(Node current, List<Book> books) {
    if (current == null) {
        return;
    }

    addBooksInOrder(current.left, books);
    books.add(current.book);
    addBooksInOrder(current.right, books);
}
}
```

Appendix C: BorrowHistoryStack Core Methods

```
public class BorrowHistoryStack {
    private Node top;

    //One linked-list node used by the stack.
    private static class Node {
        private final LoanRecord loanRecord;
        private final Node next;

        private Node(LoanRecord loanRecord, Node next) {
            this.loanRecord = loanRecord;
            this.next = next;
        }
    }

    /**
     * Adds a new loan record to the top of the stack.
     */
    public void push(LoanRecord loanRecord) {
        top = new Node(loanRecord, top);
    }

    /**
     * Checks whether the student has any stored loan records.
     */
    public boolean isEmpty() {
        return top == null;
    }

    /**
     * Finds the student's active, unreturned loan for the requested ISBN.
     */
    public LoanRecord findActiveLoanByIsbn(long isbn) {
        Node current = top;

        while (current != null) {
            LoanRecord loanRecord = current.loanRecord;
            if (!loanRecord.isReturned() && loanRecord.getBook().getIsbn() == isbn) {
                return loanRecord;
            }
            current = current.next;
        }

        return null;
    }

    /**
     * Converts the stack to a list while keeping the newest record first.
     */
    public List<LoanRecord> toList() {
        List<LoanRecord> loanRecords = new ArrayList<>();
        Node current = top;

        while (current != null) {
            loanRecords.add(current.loanRecord);
            current = current.next;
        }

        return loanRecords;
    }
}
```